

SetMyCareer – System Architecture

Information Architecture (every touchpoint) + Backend API Specification (C# / ASP.NET Core)

Version	1.0 — 18 July 2026
Status	Ready for backend implementation
Audience	Written for a developer or stakeholder who has never used the product . Part 1 explains what the product is and what every screen does. Part 2 is the server implementation your backend team integrates into <code>api.setmycareer.com</code> . They go hand-in-hand: every screen in Part 1 names the Part-2 endpoint that powers it (\$2.9).
Priority docs	Extends <code>BACKEND_API_SPEC.md</code> (the July-12 gap list) and <code>E2E_TEST_MAP.md</code> (cross-role data-flow map). Conventions are unchanged; this document is the complete, current spec.

The one rule that governs everything in Part 2: this API is **non-destructive**. Client records are never hard-deleted; test results and money movements are append-only; every admin mutation is audited. Nothing in this spec can damage the existing SetMyCareer database — new tables only, plus reads of what already exists.

Part 0 – The product in one page

SetMyCareer sells **measured career decisions**. A client takes a battery of four validated psychometric instruments, the system synthesises the results into a career report, and a human counsellor reads the results with the client across booked sessions. An AI assistant (“Compass”) is available throughout, grounded in the client’s own results.

One React codebase serves three “doors” plus a guest flow; a separate static site does marketing:

Surface	URL	Who	What it is
Client portal	<code>.../portal</code>	Students, professionals, parents	Take tests, read reports, book sessions, talk to Compass, manage package & credits
Counsellor console	<code>.../</code> (root)	SMC counsellors (“navigators”)	Caseload, calendar, live sessions w/ recording + transcript + timestamped notes, report authoring
Admin console	<code>.../admin</code>	SMC staff	Company KPIs, full client/counsellor 360°, session oversight (confirm/cancel/reschedule), access revoke, test-link generator, revenue
Guest test links	<code>.../t/<token></code>	Prospects (no account)	The full final battery (or one test via <code>?only=</code>) → on-screen report → print PDF

Surface	URL	Who	What it is
Marketing site	site-madhavs-projects-56d7586e.vercel.app	Public	Editorial site: product, pricing, blog (236 posts), resources, legal (12 docs), lead forms → Zoho

Production app: <https://setmycareer-counselor.vercel.app>

Two data planes back the app today (details §1.8):

- **Plane A — the live SMC backend** (<https://api.setmycareer.com/api/>): identity, navigators, packages, sold services, sessions, uploaded report PDFs, notes. *Already yours*.
- **Plane B — interim cloud store** (Supabase via the app's `/api/cloud`): bookings, messages, in-app test results, credits, AI chats, app state. **Part 2 of this document is the C# system that replaces Plane B on your servers** — and formalises the new product surfaces (final battery results, profiles, session transcripts, synthesis, guest links).

Demo access (for anyone reviewing the product)

Door	How to get in
Client portal	Go to <code>.../portal</code> → Create account → any name + email (no password, no card). You are in immediately, on the student journey with all gates/empty states visible. Reference identity used in testing: Demo Reviewer / <code>demo.reviewer@setmycareer.dev</code> (student track). Note: created accounts are per-browser until the C# accounts endpoint ships (§2.5-A); each reviewer just creates their own in one click.
Client portal (full live data)	Sign in with the live E2E test client (id 31369) — credentials in <code>docs/E2E_TEST_MAP.md</code> §2 (kept out of this doc deliberately).
Counsellor or console	Live navigator sign-in required — E2E test counsellor id 4104 (credentials in <code>docs/E2E_TEST_MAP.md</code>).
Admin console	Live staff credentials (same doc). No self-serve creation exists, by design.
Guest battery	No account at all: <code>.../t/smcmain01</code> (full battery), <code>.../t/smcpers01?only=personality</code> , <code>.../t/smcint01?only=interest</code> , <code>.../t/smcabil01?only=ability</code> , <code>.../t/smccomp01?only=competency</code> .

Part 1 – Information architecture

1.1 The estate map

setmycareer-counselor.vercel.app

```
|
├─ /portal ..... CLIENT DOOR (auth: OTP / email+password / 1-click create)
|   ├─ /portal/home ..... state-aware dashboard (new → assessing → active)
|   ├─ /portal/assessments ..... the 3-test battery for your track
|   |   └─ /portal/assessments/:testId · full-screen test room (brief → questions → review → submit)
|   ├─ /portal/reports ..... Reports hub + document uploads
|   |   ├─ /portal/reports/career · consolidated Career Intelligence Report
|   |   └─ /portal/reports/test/:testId · per-instrument report
|   ├─ /portal/sessions ..... planner (mini-month + day timeline), booking, recaps, transcripts
|   ├─ /portal/journey ..... dated journey stream + programme spine
|   ├─ /portal/therapy ..... Compass (AI) full-screen chat
|   ├─ /portal/voice ..... live voice session (persona + audio-reactive orb)
|   ├─ /portal/messages ..... client ↔ counsellor messaging
|   |   └─ /portal/call/:clientId · video/voice call room (in-call chat, incoming-ring overlay)
|   ├─ /portal/terminal ..... career market terminal (track-aware)
|   ├─ /portal/resources ..... editorial library (blog feed)
|   ├─ /portal/services ..... 2026 catalogue (journeys, add-ons)
|   |   └─ /portal/services/:productId · product page → checkout
|   ├─ /portal/billing ..... Package & credits (packs, guide, FAQs)
|   └─ /portal/account ..... profile intake (the gate) + account
|
├─ / ..... COUNSELLOR DOOR (live navigator sign-in + service agreement)
|   ├─ dashboard ..... caseload overview, today's sessions, notifications
|   ├─ clients + client detail ..... live caseload 360° (results, reports, notes, docs)
|   ├─ calendar ..... month/week/day/list + live session merge + blocking
|   ├─ session room ..... recording, live transcript, timestamped notes
|   ├─ library ..... recordings + transcripts
|   ├─ assistant ..... Compass (counsellor grounding)
|   ├─ terminal ..... market terminal (counsellor view)
|   └─ settings
|
├─ /admin ..... ADMIN DOOR (staff sign-in)
|   ├─ mission control ..... company KPIs, funnel, MRR, forecast
|   ├─ clients + 360° ..... every client: packages, reports, tests, transcripts, flags
|   ├─ counsellors + detail ..... roster, capacity, approvals (EnableNavigator)
|   ├─ sessions ..... ALL upcoming bookings (client↔counsellor nesting) +
|   |   └─ confirm / cancel / reschedule / portal-access revoke
|   ├─ revenue / economics / growth · money + unit economics + lifecycle
|   └─ coupons ..... Razorpay coupons + refunds
```

- | └─ test links guest-link generator (/t tokens)
- | └─ api & usage endpoint catalogue + provider usage
- |
- | └─ /t/<token> GUEST BATTERY (no auth)
- | └─ intake form → test(s) → per-test closure → consolidated report → print PDF
- |
- | └─ marketing site (separate deploy)
 - | └─ / product / pricing / counsellors / contact / blog / resources / videos / legal/*
 - | └─ every CTA lands on: /portal (sign-up), / (counsellor apply), checkout APIs

1.2 Client portal – floor plan

The portal is **state-aware**: the same routes render differently for a brand-new account (gates + invitations), a mid-battery account (progress), and an active client (live data). The typography system (`.portal-type`) and the single lime accent `#d8e94f` (live/now states only) are constant throughout.

The gate that shapes everything: profile intake

A new account can browse, but **cannot start tests or book sessions** until the profile is complete: name, age, gender, location, phone + email (autofilled from the sign-up channel), track + stage, LinkedIn (optional), and two required subjective questions (how they make decisions; what success looks like). The gate appears as: a progress dropdown on Home (“Finish your profile — 30%”), a full-screen guard if a test is opened, and a gate card on Sessions. All three link to `/portal/account`. This is deliberate: the battery’s third test and the counsellor auto-assignment both depend on profile facts.

Screen	What it is	Key actions → feedback	Data
Auth door (<code>/portal</code>)	Split-screen sign-in/sign-up. Sign in = phone OTP or email+password (live accounts). Create account = 1-click, passwordless. Wrong-door links to counsellor/admin.	Send OTP → code stage + resend; Create account → straight into Home; Google button; legal links → live legal pages	Live <code>SendOtp/Logi</code> <code>nWithOtp/Log</code> <code>inWithPasswo</code> <code>rd</code> ; local account seed
Home (<code>/portal</code> / <code>/home</code>)	State-aware dashboard: journey hero, 9-node programme spine, next-best action, market match rows, profile nudge	Every card navigates to its source page; Book session → Sessions; nudge → Account	results-store, bookings, live package, profile
Assessme nts (<code>/portal</code> / <code>/assessm</code> <code>ents</code>)	The 3 instruments for your track (personality 72, interest 176, then DBDA 245 for students / CCPA 88 for professionals) as isometric-glyph cards w/ status	Start/Resume → test room; completed → View report	catalog <code>testsFor(tra</code> <code>ck)</code> , results- store

Screen	What it is	Key actions → feedback	Data
Test room (/portal/assessments/:testId)	Full-screen one-take flow: splash brief (what it measures, how to answer, no right answers, how it feeds the report) → questions (per-client shuffled) → review → honesty submit → completion screen w/ [Next test] [Back to assessments]. Crash-safe drafts; deliberately no fullscreen-exit control .	Begin → questions; submit → completion + result saved toast; abandoned mid-test → draft restored on return	final engines (pfin/ifin/DBDA/CCPA), results-store, profile (norms)
Reports (/portal/reports)	Report hub: per-instrument cards (report or “take the test” prompt), consolidated synthesis when ready, Documents pane (upload mark-sheets etc. — visible to your counsellor + admin)	Open report; Upload document → progress + success toast (15 MB cap, title prompt)	results-store, live Reports/* rail
Test report (/portal/reports/test/:id)	Instrument-specific report: personality radar + facet interpretations; interest career-level graph + categories + hobby-gap; DBDA grades / CCPA composites	Print/PDF; links into Journey/Sessions	report-bridge (final engines)
Sessions (/portal/session s)	Full-width planner: mini-month (busy dots) + day timeline (9–18h) + upcoming rows w/ Join, sessions-left widget (number + segmented bar; lime = ready to book), request form (topic/time), past-session recaps	Request session → auto-assigned counsellor + reason toast; Join (gated to real time window); recap → Read full screen / Export .txt / Discuss with Compass	bookings store, assignment engine, live sold services
Journey (/portal/journey)	The dated journey stream (tests w/ standout factor, session summaries, purchases, upcoming w/ Join now) + programme spine	Every widget → its source page	results, bookings, purchases
Compass (/portal/therapy)	Full-screen AI chat; generative-UI cards (career, package, compare); grounded in the client’s own results + career patterns; credit-metered. Also everywhere as the floating bar (⌘K).	Send → streamed answer + cards; card actions (book, open report); low credits → top-up chip	/api/assistant , chats cloud store, credit ledger
Voice (/portal/voice)	Live voice session: persona + voice picker, audio-reactive orb, live chat-format transcript; burns voice credits by the minute	Preview voice; start/end session → orb + transcript states	browser STT/TTS + /api/assistant
Messages (/portal/messages)	Client ↔ counsellor thread w/ attachments and resource tags; call button → call room (/portal/call/:clientId) w/ incoming-ring on the other side	Send → optimistic append + sync; Accept/Decline ring	messages store (cloud), LiveKit
Terminal (/portal/terminal)	Track-aware career market terminal: signals, news, careers	Search, open career pages	terminal data + news feed

Screen	What it is	Key actions → feedback	Data
Resources / Services / Product / Billing	Library (blog feed); 2026 catalogue (track switch); product page (/portal/services/:productId) w/ checkout rail + coupons; Package & credits w/ packs + credits guide + FAQs	Buy → “Opening secure checkout...” → Razorpay modal → server HMAC verify → grant toast + portal grants	products catalog, /api/razorpay , purchases
Account (/portal /account)	The profile intake (numbered editorial form on white) + package row + sign-out	Save → completeness updates everywhere instantly	portal-store profile

1.3 Counsellor console – floor plan

Gated twice: a **live navigator sign-in** (no local accounts) and, before any client work, a one-time **service agreement** overlay — profile confirmation + three explicit accepts (T&C, DPDP confidentiality, professional conduct). Declining keeps the console locked.

Screen	What it is	Key actions → feedback
Dashboard	Today at a glance: caseload, upcoming sessions, live-call banner, notifications	Start/join session → session room; notification → source
Clients	Live caseload; client detail = the full 360°: profile, test results, reports (incl. client-uploaded documents), notes, messages	Open report/doc → viewer; note → saved chip
Calendar	Google-Calendar-grade: month/week/day/list, drag-reschedule, event CRUD, live SMC sessions folded in read-only, blocking via ordinary events	Create/edit/delete event → optimistic + undo toast; Join → LiveKit room
Session room	Recording w/ persistent waveform, live transcript (real STT only), timestamped notes, in-call chat + client calendar	Record/stop, note at timestamp → pinned; end → duration logged
Client detail tabs (/clients/:id/...)	Per-client: overview, sessions + session detail, transcripts + approval review (approve/adjust per delta), tests + psychometric report doc (print/PDF), notes, reports	Approve & share → “Shared with client” toast; save note → validation toast
Reports hub + builder (/reports , /reports/new , /reports/preview)	The counsellor-authored Career Intelligence Report : 5-step wizard (client, inputs, synthesis) → editable doc (Tiptap) → share to the client’s portal + print/PDF	Stepper fills; Edit toggle banner; Share → toast; client sees it under Reports
Library	All recordings + transcripts, filter by client	Open/export
Assistant	Compass with counsellor grounding (methodology, caseload context)	Chat/actions
Terminal / Settings	Market terminal; appearance + account settings	—

1.4 Admin console – floor plan

Screen	What it is	Key actions → feedback
Mission Control (/admin)	Company bento: MRR waterfall, funnel, cohorts, forecast, health worklist, live client activity	Refresh (spinner + skeletons); drill-through to every section
Clients (/admin/clients , /admin/clients/:id)	Directory (live search + category filter) + full 360° per client — seeded and live-id variants (packages, reports incl. uploads, tests, transcripts, activity, flags)	Add/pause/reactivate/archive (state chip updates), assign, schedule → modal closes + row appears
Counsellors (/admin/counsellors , /:id)	Roster + detail (capacity, load)	Pending-approval banner → applications queue
Applications (/admin/applications)	Expert-application approval queue	Approve (“Working...” → green flash), Hold
Sessions (/admin/sessions)	All upcoming bookings nested client ↔ counsellor ↔ time ↔ status; assignment reason visible	Confirm / Cancel / Reschedule (toast; status back to “requested” for re-confirm) / Revoke portal access (signs the client out)
Revenue (/admin/revenue) + Economics + Growth + Journeys + Marketing	Money w/ 2026-vs-Legacy catalogue toggle, unit economics, lifecycle, funnel, ad spend	Segmented toggles, refresh spinners, export
Coupons & refunds (/admin/commerce)	Real Razorpay coupons + refunds ledger	New coupon → row appears; Active/Paused chip flips; refund → Razorpay-confirmed toast
Test links (/admin/test-links)	Guest-link generator: mint /t/<token> links (1–200 at once, full battery or ?only=), copy modal, batch sheet	Generate → copy modal; per-row Copy → “Link copied” toast
API & usage (/admin/api)	Endpoint catalogue + provider usage + Core-API connection check	Check connection → pulsing status dot
Assistant (/admin/assistant)	Admin AI copilot (operator prompts, quick actions)	Streamed replies
Reports / Access / Settings	Transcript+report browser, roles, settings	—

1.5 Guest test links (/t/<token>)

No account, no payment. Intake form (name, age, gender, grade/track — **student vs executive decides the third test**) → the battery (personality 72 → interest 176 → DBDA **or** CCPA; ?only= scopes to one instrument) → encouraging closure per test → consolidated on-screen report (Gemini synthesis via /api/consolidate) → print-to-PDF. **Start over** chip resets the token’s local state. Results live only in that browser today — §2.5-H gives them a home.

1.6 Marketing site → product wiring

The site is a full funnel, not a brochure — it has its own sign-in and checkout that ride the app’s APIs. All wiring verified live (HTTP 200) on 18 Jul 2026:

Site page / CTA	Target
Sign in / Start / “Open the app” (nav, hero, /solutions , blog CTAs)	.../portal (client door)
Site’s own /signin (OTP or password) + /checkout/:tierId (coupon + pay rail)	app APIs: live login + .../api/razorpay (order + HMAC verify)
Free tests: /cri (Career Clarity Index, 20 statements) and /fit (package-fit quiz)	on-site reports → portal/purchase CTAs; .../api/fit-report
Counsellor recruiting (/counsellors) → /experts/apply	application form → admin approval queue (/admin/applications)
Site chatbot (CareerBar)	.../api/assistant
Contact / lead forms / e-book	/api/lead → Zoho FormTracker; e-book via Resend
VCLP programmes (/programs/:slug)	application form (done-state confirmation)
Legal footer (12 docs, /legal/*)	also linked from the portal sign-up consent line

1.7 The three lifecycles (how the product actually flows)

Client — Marketing page → /portal create account (or OTP sign-in) → **profile intake** (gate) → battery of 3 (third auto-picked by track) → per-test reports appear → synthesis → counsellor **auto-assigned** on first session request (expertise + availability + profile keywords; the reason is shown to the client and admin) → sessions (join → live room → transcript + notes) → recaps readable/exportable/discussable with Compass → Journey stream accretes every event → more purchases extend the spine.

Counsellor — apply on the site → admin approves → sign in → **service agreement** → caseload appears → calendar (block time, reschedule) → run sessions (record, transcript, timestamped notes) → notes/report shared to client → Library archives everything.

Admin — sign in → Mission Control → oversee every booking (confirm/cancel/reschedule), revoke portal access when needed, mint guest test links, manage coupons/refunds, watch revenue + capacity + API usage. Every mutation is toast-confirmed in the UI and (in Part 2) lands in the audit log.

1.8 Data planes today → target

Data	Today	Target (Part 2)
Identity, navigators, packages, sold services, live sessions, report PDFs, notes	Plane A — live api.setmycareer.com	unchanged (already yours)
Bookings, messages, credits, AI chats, app state	Plane B — Supabase via app /api/cloud	C# API (§2.5 A–F) — swap point is a single file (src/lib/cloud.ts)
Final-battery results + payloads	browser + cloud digest	C# API Tests/* (§2.5-C) — append-only
Profile intake	browser + cloud	C# API Profile/* (§2.5-B)
Session transcripts + timestamped notes	browser (counsellor) / cloud	C# API Sessions/* (§2.5-D)
Synthesis (LangGraph report)	Railway FastAPI, results ephemeral	C# API Synthesis/* (§2.5-E) — jobs + stored results
Guest link results	that browser only	C# API GuestLinks/* (§2.5-H)
Admin flags (revoke), agreements	localStorage	C# API Admin/* , Counsellor/* (§2.5-F/G)

Part 2 – Backend API specification (C# / ASP.NET Core 8)

2.0 Design doctrine (read first)

1. Non-destructive, always.
 - **No client is ever hard-deleted.** There is no `DELETE` verb on any client-bearing resource. “Remove” = `IsArchived = true` (admin-only, audited, reversible).
 - **Append-only ledgers:** `TestResults` , `CreditLedger` , `AuditEvents` , `SynthesisJobs` have no update or delete endpoints. A retake is a **new row**; “latest” is `MAX(TakenAtUtc)` . History is the product (progress over time), so history is never lost.
 - **Write once to add, write again to update** — creates are `POST` , updates are `PUT` with optimistic concurrency (`RowVersion`); no endpoint ever silently overwrites newer data.

2. **New tables only.** Nothing here alters existing SetMyCareer tables. Reads of existing data go through the endpoints you already run (`User/*` , `NavigatorDetail/*` , `Reports/*` , `DedicatedServices/*` ...). The new controllers live beside them.
 3. **Conventions match your existing API** (per `BACKEND_API_SPEC.md §1`): base `https://api.setmycareer.com/api/` , routes `api/{Controller}/{Action}` , JSON, numeric ids sent as strings, envelope `{ "success": true, "data": ... } / { "success": false, "message": "..." }` , CORS for the two app origins.
 4. **AuthN/AuthZ:** JWT bearer (issued by the existing logins, upgraded per §2.8) with roles `Client` , `Counsellor` , `Admin` . Every action below is annotated. A client can only read or write records keyed to their own `UserId` ; counsellors only their caseload; admin everything. Passwords: **BCrypt** at rest (§2.8).
 5. **Secrets** are configuration **names**, never values, and never in the repo (§2.7).
-

2.1 Solution layout

```
Smc.Api/
├─ Program.cs                // DI, auth, CORS, Swagger, migrations
├─ Domain/Entities.cs        // §2.2 – all entities
├─ Data/SmcDbContext.cs      // §2.3 – EF Core, soft-delete filter, indexes
├─ Contracts/Dtos.cs         // §2.4 – request/response DTOs
├─ Controllers/              // §2.5 – one controller per area (A–H)
├─ Services/SynthesisOrchestrator.cs // §2.6 – LangGraph bridge
├─ Services/RazorpayVerifier.cs // HMAC signature verification
├─ Infrastructure/Audit.cs    // audit-event writer (called by every admin mutation)
└─ appsettings.json          // §2.7 – env-var-driven template
```

EF Core provider: use whatever the existing DB runs (`UseSqlServer` / `UseNpgsql`) — the model below is provider-agnostic. Run `dotnet ef migrations add SmcAppTables` once; the migration only **creates** new tables.

2.2 Domain entities

```
// Domain/Entities.cs
namespace Smc.Api.Domain;

public abstract class Row
{
    public long Id { get; set; }
    public DateTime CreatedAtUtc { get; set; } = DateTime.UtcNow;
    public DateTime UpdatedAtUtc { get; set; } = DateTime.UtcNow;
    [System.ComponentModel.DataAnnotations.Timestamp] public byte[]? RowVersion { get; set; }
}

/// Client-side profile intake. One row per client (existing numeric user id).
public class ClientProfile : Row
{
    public string UserId { get; set; } = ""; // existing SMC numeric id, as string
    public string FullName { get; set; } = "";
    public int? Age { get; set; }
    public string? Gender { get; set; }
    public string? Location { get; set; }
    public string Email { get; set; } = "";
    public string Phone { get; set; } = "";
    public string? LinkedIn { get; set; }
    public string Track { get; set; } = "student"; // student | professional
    public string? Stage { get; set; } // one of PROFILE_STAGES (frontend enum)
    public string? QDecision { get; set; } // required subjective answers
    public string? QSuccess { get; set; }
    public string? SubjectiveJson { get; set; } // remaining optional answers, verbatim
    public DateTime? CompletedAtUtc { get; set; } // set once requirements are met
}

/// Append-only. One row per completed instrument run (portal battery).
public class TestResult : Row
{
    public string UserId { get; set; } = "";
    public string TestId { get; set; } = ""; // sigma_personality | sigma_interest |
    aptitude
    public string? Variant { get; set; } // null | dbda | ccpa (third test, by track)
    public string EngineVersion { get; set; } = "final-2026-07";
    public DateTime TakenAtUtc { get; set; }
    public string ScoresJson { get; set; } = "{}"; // flat factor→0-100 map (chart-ready)
```

```

    public double Overall { get; set; }
    public string? PayloadJson { get; set; }           // full engine output (facets, grades,
composites)
    public string? AnswersJson { get; set; }           // raw answers at ORIGINAL item index
}

/// Session booking – the single session entity the whole system converges on.
public class Booking : Row
{
    public string ClientUserId { get; set; } = "";
    public string? CounsellorUserId { get; set; }      // null until assigned
    public string? AssignReason { get; set; }          // human-readable auto-assign rationale
    public DateTime AtUtc { get; set; }
    public int DurationMin { get; set; } = 60;
    public string Topic { get; set; } = "";
    public string Status { get; set; } = "requested"; //
requested|confirmed|completed|canceled|no_show
    public string CreatedByRole { get; set; } = "";   // client | counsellor | admin
    public string RoomName { get; set; } = "";        // LiveKit: "smc-<clientUserId>"
    public int? ActualMin { get; set; }
}

/// Timestamped note inside a session. Append-only per booking.
public class SessionNote : Row
{
    public long BookingId { get; set; }
    public int AtSec { get; set; }
    public string Text { get; set; } = "";
    public string AuthorRole { get; set; } = "counsellor";
}

/// One transcript per booking (replaceable until the booking is completed, then frozen).
public class SessionTranscript : Row
{
    public long BookingId { get; set; }
    public string Text { get; set; } = "";
    public string Source { get; set; } = "live_stt";   // live_stt | upload
    public string? Language { get; set; }
}

/// Counsellor service agreement – required before offering services.
public class CounsellorAgreement : Row
{

```

```

    public string CounsellorUserId { get; set; } = "";
    public string Version { get; set; } = "v1";
    public bool AcceptedTerms { get; set; }
    public bool AcceptedConfidentiality { get; set; } // DPDP
    public bool AcceptedConduct { get; set; }
    public string ProfileSnapshotJson { get; set; } = "{}";
    public DateTime AcceptedAtUtc { get; set; }
}

/// Calendar block (counsellor unavailability) – feeds auto-assignment + admin calendars.
public class CounsellorBlock : Row
{
    public string CounsellorUserId { get; set; } = "";
    public DateTime StartUtc { get; set; }
    public DateTime EndUtc { get; set; }
    public string? Reason { get; set; }
    public bool IsArchived { get; set; } // cancel = archive, never delete
}

/// Append-only money/credit movement. Balance = SUM(Delta) per (UserId, Kind).
public class CreditLedgerEntry : Row
{
    public string UserId { get; set; } = "";
    public string Kind { get; set; } = ""; // career | voice | session
    public int Delta { get; set; } // + grant, - burn
    public string Reason { get; set; } = ""; // purchase | chat | voice_minute | booking |
admin_grant
    public string? RefId { get; set; } // razorpay payment id / booking id / chat id
}

/// A verified purchase (Razorpay). Status never goes backwards; refund is a new fact, not a delete.
public class Purchase : Row
{
    public string UserId { get; set; } = "";
    public string ProductId { get; set; } = ""; // catalog id (sj_/pro_/mk_/credit pack)
    public string Label { get; set; } = "";
    public long AmountPaise { get; set; }
    public string RazorpayOrderId { get; set; } = "";
    public string? RazorpayPaymentId { get; set; }
    public bool SignatureVerified { get; set; } // HMAC verified server-side – REQUIRED for
grants
    public string Status { get; set; } = "created"; // created | paid | refunded
}

```

```

/// Admin flag: portal access revoke. The portal signs a revoked client out on load.
public class PortalAccessFlag : Row
{
    public string UserId { get; set; } = "";
    public bool Revoked { get; set; }
    public string? Reason { get; set; }
    public string ByAdminUserId { get; set; } = "";
}

/// Generic per-user app state (replaces Supabase app_state). Key examples:
/// "shared.bookings", "calendar-events", "wallet", "portal.profile.draft".
public class AppState : Row
{
    public string App { get; set; } = ""; // client | counsellor | admin
    public string UserId { get; set; } = "";
    public string Key { get; set; } = "";
    public string ValueJson { get; set; } = "";
}

/// AI chat threads (replaces Supabase app_chats). Cap 50 per (App, UserId): on insert
/// beyond the cap, archive the oldest – do not delete.
public class ChatThread : Row
{
    public string App { get; set; } = "";
    public string UserId { get; set; } = "";
    public string ChatId { get; set; } = ""; // client-generated uuid
    public string Title { get; set; } = "";
    public string MessagesJson { get; set; } = "[]";
    public bool IsArchived { get; set; }
}

/// Client ↔ counsellor messaging.
public class Message : Row
{
    public string ThreadKey { get; set; } = ""; // "<clientId>:<counsellorId>"
    public string SenderRole { get; set; } = ""; // client | counsellor
    public string Text { get; set; } = "";
    public string? AttachmentUrl { get; set; }
}

/// Guest test links + their results (today: browser-only).
public class GuestTestLink : Row

```

```

{
    public string Token { get; set; } = "";           // e.g. smcmain01
    public string Mode { get; set; } = "full";        // full | only:personality | only:interest |
only:ability | only:competency
    public string? Label { get; set; }
    public string CreatedByUserId { get; set; } = "";
    public bool Active { get; set; } = true;          // deactivate, never delete
}

public class GuestTestResult : Row                    // append-only
{
    public string Token { get; set; } = "";
    public string IntakeJson { get; set; } = "{}";    // name/age/gender/track from the guest form
    public string TestId { get; set; } = "";
    public string? Variant { get; set; }
    public string ScoresJson { get; set; } = "{}";
    public string? PayloadJson { get; set; }
    public DateTime TakenAtUtc { get; set; }
}

/// Synthesis job – one LangGraph run over a client's full record. Append-only.
public class SynthesisJob : Row
{
    public string UserId { get; set; } = "";
    public string Status { get; set; } = "queued";    // queued | running | done | failed
    public string InputSnapshotJson { get; set; } = "{}"; // the exact inputs sent (reproducibility)
    public string? ResultJson { get; set; }           // the counsellor report / blueprint
    public string? Error { get; set; }
    public string EngineVersion { get; set; } = "";   // LangGraph service version header
    public DateTime? StartedAtUtc { get; set; }
    public DateTime? FinishedAtUtc { get; set; }
}

/// Every admin/counsellor mutation lands here. Append-only, queryable in /admin.
public class AuditEvent : Row
{
    public string ActorUserId { get; set; } = "";
    public string ActorRole { get; set; } = "";
    public string Action { get; set; } = "";          // e.g. booking.reschedule, portal.revoke
    public string Entity { get; set; } = "";
    public string EntityId { get; set; } = "";
    public string DetailJson { get; set; } = "{}";
}

```

2.3 DbContext

```
// Data/SmcDbContext.cs
using Microsoft.EntityFrameworkCore;
using Smc.Api.Domain;

namespace Smc.Api.Data;

public class SmcDbContext(DbContextOptions<SmcDbContext> o) : DbContext(o)
{
    public DbSet<ClientProfile> Profiles => Set<ClientProfile>();
    public DbSet<TestResult> TestResults => Set<TestResult>();
    public DbSet<Booking> Bookings => Set<Booking>();
    public DbSet<SessionNote> SessionNotes => Set<SessionNote>();
    public DbSet<SessionTranscript> SessionTranscripts => Set<SessionTranscript>();
    public DbSet<CounsellorAgreement> CounsellorAgreements => Set<CounsellorAgreement>();
    public DbSet<CounsellorBlock> CounsellorBlocks => Set<CounsellorBlock>();
    public DbSet<CreditLedgerEntry> CreditLedger => Set<CreditLedgerEntry>();
    public DbSet<Purchase> Purchases => Set<Purchase>();
    public DbSet<PortalAccessFlag> PortalAccessFlags => Set<PortalAccessFlag>();
    public DbSet<AppState> AppState => Set<AppState>();
    public DbSet<ChatThread> ChatThreads => Set<ChatThread>();
    public DbSet<Message> Messages => Set<Message>();
    public DbSet<GuestTestLink> GuestTestLinks => Set<GuestTestLink>();
    public DbSet<GuestTestResult> GuestTestResults => Set<GuestTestResult>();
    public DbSet<SynthesisJob> SynthesisJobs => Set<SynthesisJob>();
    public DbSet<AuditEvent> AuditEvents => Set<AuditEvent>();

    protected override void OnModelCreating(ModelBuilder b)
    {
        b.Entity<ClientProfile>().HasIndex(x => x.UserId).IsUnique();
        b.Entity<TestResult>().HasIndex(x => new { x.UserId, x.TestId, x.TakenAtUtc });
        b.Entity<Booking>().HasIndex(x => new { x.ClientUserId, x.AtUtc });
        b.Entity<Booking>().HasIndex(x => new { x.CounsellorUserId, x.AtUtc });
        b.Entity<SessionNote>().HasIndex(x => x.BookingId);
        b.Entity<SessionTranscript>().HasIndex(x => x.BookingId).IsUnique();
        b.Entity<CreditLedgerEntry>().HasIndex(x => new { x.UserId, x.Kind });
        b.Entity<Purchase>().HasIndex(x => x.RazorpayOrderId).IsUnique();
        b.Entity<PortalAccessFlag>().HasIndex(x => x.UserId).IsUnique();
        b.Entity<AppState>().HasIndex(x => new { x.App, x.UserId, x.Key }).IsUnique();
        b.Entity<ChatThread>().HasIndex(x => new { x.App, x.UserId, x.ChatId }).IsUnique();
        b.Entity<Message>().HasIndex(x => new { x.ThreadKey, x.CreatedAtUtc });
    }
}
```



```

        b.Entity<GuestTestLink>().HasIndex(x => x.Token).IsUnique();
        b.Entity<GuestTestResult>().HasIndex(x => x.Token);
        b.Entity<SynthesisJob>().HasIndex(x => new { x.UserId, x.CreatedAtUtc });
        b.Entity<AuditEvent>().HasIndex(x => new { x.Entity, x.EntityId });

        // Soft-archive filters – archived rows vanish from normal queries but are never gone.
        b.Entity<CounsellorBlock>().HasQueryFilter(x => !x.IsArchived);
        b.Entity<ChatThread>().HasQueryFilter(x => !x.IsArchived);
    }

    public override Task<int> SaveChangesAsync(CancellationToken ct = default)
    {
        foreach (var e in ChangeTracker.Entries<Row>())
            if (e.State == EntityState.Modified) e.Entity.UpdatedAtUtc = DateTime.UtcNow;
        return base.SaveChangesAsync(ct);
    }
}

```

2.4 Response envelope + core DTOs

```
// Contracts/Dtos.cs
namespace Smc.Api.Contracts;

public record Envelope<T>(bool Success, T? Data, string? Message = null)
{
    public static Envelope<T> Ok(T data) => new(true, data);
    public static Envelope<T> Fail(string message) => new(false, default, message);
}

public record ProfileUpsertReq(string FullName, int? Age, string? Gender, string? Location,
    string Email, string Phone, string? LinkedIn, string Track, string? Stage,
    string? QDecision, string? QSuccess, string? SubjectiveJson);

public record TestSubmitReq(string TestId, string? Variant, DateTime TakenAtUtc,
    string ScoresJson, double Overall, string? PayloadJson, string? AnswersJson);

public record BookingCreateReq(DateTime AtUtc, int DurationMin, string Topic,
    string? PreferredCounsellorUserId);

public record BookingAdminActionReq(string BookingId, DateTime? NewAtUtc, string? Reason);

public record CheckoutCreateReq(string ProductId); // amount from SERVER catalog, never
client
public record CheckoutVerifyReq(string RazorpayOrderId, string RazorpayPaymentId, string
    RazorpaySignature);

public record StateSetReq(string Key, string ValueJson);
public record ChatUpsertReq(string ChatId, string Title, string MessagesJson);
public record MessageSendReq(string ThreadKey, string Text, string? AttachmentUrl);
public record GuestResultReq(string Token, string IntakeJson, string TestId, string? Variant,
    string ScoresJson, string? PayloadJson, DateTime TakenAtUtc);
public record AgreementReq(bool AcceptedTerms, bool AcceptedConfidentiality, bool AcceptedConduct,
    string ProfileSnapshotJson);
public record BlockReq(DateTime StartUtc, DateTime EndUtc, string? Reason);
public record RevokeReq(string UserId, bool Revoked, string? Reason);
```

2.5 Controllers – the full endpoint surface

Every route is `POST / GET / PUT` under `api/{Controller}/{Action}` ; all return the envelope.

`[Authorize(Roles=...)]` shown per action. **There are no DELETE endpoints anywhere in this spec** — that is intentional and must survive code review.

A. `AccountsController` – portal accounts (closes the local-only sign-up gap)

Action	Verb	Role	Behaviour
<code>Create</code>	<code>POST</code>	anon	<code>{name, email, track, goal?}</code> → creates a real backend user (existing user table via your <code>AddUser</code> path or equivalent), seeds <code>ClientProfile</code> , returns <code>{userId, jwt}</code> . Idempotent on email: an existing email returns <code>success:false, "account exists – sign in"</code> .
<code>SendMagicLink</code>	<code>POST</code>	anon	emails a one-time sign-in link (so a created account works from any device — completes the portal’s “magic link” promise).
<code>Me</code>	<code>GET</code>	Client	profile + credits + revoke flag in one call (the portal boot payload).

B. `ProfileController`

Action	Verb	Role	Behaviour
<code>Get</code>	<code>GET</code>	Client (own) / Counsellor (caseload) / Admin	the intake record + completeness %.
<code>Upsert</code>	<code>PUT</code>	Client	<code>ProfileUpsertReq</code> → create-or-update own row; server recomputes <code>CompletedAtUtc</code> when required fields present (mirrors the portal’s <code>profileRequirements</code>). Gates on the frontend read this.

C. `TestsController` – append-only

Action	Verb	Role	Behaviour
<code>Submit</code>	<code>POST</code>	Client	<code>TestSubmitReq</code> → new <code>TestResult</code> row. Never upserts: a retake is a new row. Rejects unknown <code>TestId</code> . Fires <code>Synthesis/Enqueue</code> automatically when the battery (3 distinct <code>TestIds</code>) completes.
<code>ListMine</code>	<code>GET</code>	Client	all rows, newest first (the portal shows latest per instrument; history feeds progress-over-time).

Action	Verb	Role	Behaviour
ListForClient	GET	Counselor/Admin	?userId= — same shape, for the 360°.

D. SessionsController

Action	Verb	Role	Behaviour
Request	POST	Client	BookingCreateReq → status requested ; if no preferred counsellor, runs auto-assignment (expertise keywords from the client profile × navigator practicing_expertise , minus upcoming-load penalty — the exact ranking in src/portal/assignment.ts), stores AssignReason .
MyBookings	GET	Client	own bookings + notes count + transcript flag.
Caseload	GET	Counsellor	own upcoming + past bookings.
Confirm / Cancel / Complete	POST	Counsellor/Admin	status transitions (state machine: requested→confirmed→completed; any→canceled; confirmed→no_show). Invalid transitions are rejected — never silently forced.
Reschedule	POST	Admin/Counsellor	BookingAdminActionReq → new time, status back to requested (client re-confirms), audited.
AddNote	POST	Counsellor	timestamped note (append-only).
Notes / Transcript	GET	Client (own) / Counsellor / Admin	the recap surfaces (portal “Read full screen” + “Export .txt” read these).
SaveTranscript	PUT	Counsellor	upsert until the booking is completed , then frozen (409 afterwards).

E. SynthesisController – the LangGraph bridge (see §2.6)

Action	Verb	Role	Behaviour
Enqueue	POST	Client/Counsellor/Admin	creates a SynthesisJob (queued) snapshotting the client’s profile + latest 3 results (+ optional transcript), returns jobId . Dedupes: an identical pending snapshot returns the existing job.

Action	Verb	Role	Behaviour
Status	GET	same	?jobId= → status + error. Poll or subscribe.
Result	GET	same	the stored ResultJson (the counsellor report).
Latest	GET	same	newest done job for a user — what the portal Reports hub and the counsellor 360° render.

F. AdminController

Action	Verb	Role	Behaviour
AllUpcomingBookings	GET	Admin	every booking, nested client↔counsellor, incl. AssignReason — the oversight panel.
RevokePortalAccess	POST	Admin	RevokeReq → upsert PortalAccessFlag + audit. The portal checks this on boot and signs the client out (today localStorage-only; this makes it cross-device).
ArchiveClient	POST	Admin	soft-archive only; audited; reversible. This is the only “remove client” that exists.
AuditTrail	GET	Admin	filter by entity/actor/date.
Calendars	GET	Admin	merged view: bookings + counsellor blocks, per counsellor per day.

G. CounsellorController

Action	Verb	Role	Behaviour
Agreement	GET/P OST	Counsellor	read / accept the service agreement (all three accepts required; version-stamped). Console gates on this — server-side it also gates Confirm / AddNote (403 until accepted).
Blocks	GET/P OST	Counsellor	list / create calendar blocks; CancelBlock = archive. Auto-assignment and Admin/Calendars respect them.

H. StateController , ChatsController , MessagesController , GuestLinksController

Direct ports of the interim cloud store + guest flow (contracts already exercised by the app):

- State/GetAll?app=&userId= · State/Set (StateSetReq) · State/Remove (sets tombstone, keeps history)
- Chats/List · Chats/Upser (ChatUpserReq , cap 50 → archive oldest) · Chats/Archive
- Messages/Thread?threadKey= · Messages/Send (MessageSendReq)

- `GuestLinks/Create` (Admin) · `GuestLinks/Get?token=` (anon — validates + returns mode) · `GuestLinks/SubmitResult` (anon, `GuestResultReq`, append-only) · `GuestLinks/Deactivate` (Admin)
- `Checkout/Create` (`CheckoutCreateReq` — price from the **server** catalog) · `Checkout/Verify` (`CheckoutVerifyReq` — HMAC `order_id|payment_id` with the Razorpay secret; only a verified signature writes `Purchase(paid)` + `CreditLedger` grants). Mirrors the app's existing `/api/razorpay` serverless flow so either can fulfil.

Reference implementation – two controllers in full

```
// Controllers/TestsController.cs
[ApiController, Route("api/[controller]/[action]")]
public class TestsController(SmcDbContext db, SynthesisOrchestrator synth) : ControllerBase
{
    [HttpPost, Authorize(Roles = "Client")]
    public async Task<Envelope<object>> Submit(TestSubmitReq req)
    {
        var userId = User.FindFirstValue("uid")!;
        var known = new[] { "sigma_personality", "sigma_interest", "aptitude" };
        if (!known.Contains(req.TestId)) return Envelope<object>.Fail("unknown testId");

        db.TestResults.Add(new TestResult {
            UserId = userId, TestId = req.TestId, Variant = req.Variant,
            TakenAtUtc = req.TakenAtUtc, ScoresJson = req.ScoresJson,
            Overall = req.Overall, PayloadJson = req.PayloadJson, AnswersJson = req.AnswersJson,
        });
        await db.SaveChangesAsync();

        // battery complete (3 distinct instruments) → queue synthesis, exactly once
        var distinct = await db.TestResults.Where(t => t.UserId == userId)
            .Select(t => t.TestId).Distinct().CountAsync();
        if (distinct >= 3) await synth.EnqueueIfNew(userId);

        return Envelope<object>.Ok(new { stored = true });
    }

    [HttpGet, Authorize(Roles = "Client")]
    public async Task<Envelope<List<TestResult>>> ListMine()
    {
        var userId = User.FindFirstValue("uid")!;
        return Envelope<List<TestResult>>.Ok(await db.TestResults
            .Where(t => t.UserId == userId).OrderByDescending(t => t.TakenAtUtc).ToListAsync());
    }

    [HttpGet, Authorize(Roles = "Counsellor,Admin")]
    public async Task<Envelope<List<TestResult>>> ListForClient(string userId) =>
        Envelope<List<TestResult>>.Ok(await db.TestResults
            .Where(t => t.UserId == userId).OrderByDescending(t => t.TakenAtUtc).ToListAsync());
}
```

```
// Controllers/SessionsController.cs (state machine core)
[ApiController, Route("api/[controller]/[action]")]
public class SessionsController(SmcDbContext db, Audit audit) : ControllerBase
{
    static readonly Dictionary<string, string[]> Allowed = new() {
        ["requested"] = ["confirmed", "canceled"],
        ["confirmed"] = ["completed", "canceled", "no_show", "requested"], // requested = reschedule
        ["completed"] = [], ["canceled"] = [], ["no_show"] = [],
    };

    async Task<Envelope<Booking>> Transition(string bookingId, string to, DateTime? newAt, string?
reason)
    {
        var b = await db.Bookings.FindAsync(long.Parse(bookingId));
        if (b is null) return Envelope<Booking>.Fail("not found");
        if (!Allowed[b.Status].Contains(to)) return Envelope<Booking>.Fail($"cannot {b.Status} →
{to}");
        if (newAt is not null) b.AtUtc = newAt.Value;
        b.Status = to;
        await audit.Log(User, $"booking.{to}", "Booking", bookingId, new { newAt, reason });
        await db.SaveChangesAsync();
        return Envelope<Booking>.Ok(b);
    }

    [HttpPost, Authorize(Roles = "Counsellor,Admin")]
    public Task<Envelope<Booking>> Confirm(BookingAdminActionReq r) => Transition(r.BookingId,
"confirmed", null, r.Reason);
    [HttpPost, Authorize(Roles = "Counsellor,Admin")]
    public Task<Envelope<Booking>> Cancel(BookingAdminActionReq r) => Transition(r.BookingId,
"canceled", null, r.Reason);
    [HttpPost, Authorize(Roles = "Counsellor,Admin")] // reschedule → back to requested for re-
confirm
    public Task<Envelope<Booking>> Reschedule(BookingAdminActionReq r) => Transition(r.BookingId,
"requested", r.NewAtUtc, r.Reason);
}
```

2.6 LangGraph synthesis integration

The report brain is the **existing Python FastAPI + LangGraph service** (repo root `app/`, deployed on Railway, started via `uvicorn app.main:app`). Its graph (`app/career/graph.py`) runs seven agents in sequence:


```
labor_retriever → career_metrics → evidence_verifier → behavioral_scientist  
→ contradiction_agent → synthesis → counsellor_report
```

The C# API **orchestrates** it — it never re-implements the logic:

```

// Services/SynthesisOrchestrator.cs
public class SynthesisOrchestrator(SmcDbContext db, IHttpClientFactory http, IConfiguration cfg)
{
    public async Task<long> EnqueueIfNew(string userId)
    {
        if (await db.SynthesisJobs.AnyAsync(j => j.UserId == userId &&
            (j.Status == "queued" || j.Status == "running")))
            return (await db.SynthesisJobs.FirstAsync(j => j.UserId == userId &&
                (j.Status == "queued" || j.Status == "running"))).Id;

        var profile = await db.Profiles.FirstOrDefaultAsync(p => p.UserId == userId);
        var latest = await db.TestResults.Where(t => t.UserId == userId)
            .GroupBy(t => t.TestId).Select(g => g.OrderByDescending(t => t.TakenAtUtc).First())
            .ToListAsync();

        var job = new SynthesisJob {
            UserId = userId,
            InputSnapshotJson = JsonSerializer.Serialize(new { profile, results = latest }),
        };
        db.SynthesisJobs.Add(job);
        await db.SaveChangesAsync();
        _ = RunAsync(job.Id); // fire-and-forget; use a hosted queue in prod
        return job.Id;
    }

    async Task RunAsync(long jobId)
    {
        var job = await db.SynthesisJobs.FindAsync(jobId);
        job!.Status = "running"; job.StartedAtUtc = DateTime.UtcNow;
        await db.SaveChangesAsync();
        try
        {
            var client = http.CreateClient("langgraph"); // BaseAddress = LangGraph:BaseUrl
            // Endpoint contract: POST /api/career/report/specialised
            // body { career_profile, transcript?, mh_context? } → the blueprint report JSON.
            // Streaming variant for UIs: POST /api/career/analyze/stream (SSE "updates").
            var res = await client.PostAsJsonAsync("/api/career/report/specialised",
                JsonSerializer.Deserialize<object>(job.InputSnapshotJson));
            res.EnsureSuccessStatusCode();
            job.ResultJson = await res.Content.ReadAsStringAsync();
            job.Status = "done";
        }
        catch (Exception ex) { job.Status = "failed"; job.Error = ex.Message; }
    }
}

```

```

        job.FinishedAtUtc = DateTime.UtcNow;
        await db.SaveChangesAsync();
    }
}

```

Where the AI keys live: the LLM keys (GROQ_API_KEY , OPENROUTER_API_KEY , LLM_MODEL , STT_MODEL) are environment variables **on the Python service** (Railway), read by app/config.py . The C# side needs only LangGraph:BaseUrl (+ optional shared LangGraph:ApiKey header if you enable one). The Gemini key used by the guest consolidator (GEMINI_API_KEY) stays on the Vercel serverless side. No AI key ever ships to a browser.

2.7 Configuration & API keys

appsettings.json is a **template of names**. Every secret arrives via environment variables / your secret store — values are never committed anywhere.

```

{
  "ConnectionStrings": { "Smc": "" },           // env: SMC_DB_CONNECTION
  "Jwt": { "Issuer": "api.setmycareer.com", "Secret": "" }, // env: SMC_JWT_SECRET (256-bit)
  "Razorpay": { "KeyId": "", "KeySecret": "" }, // env: RAZORPAY_KEY_ID / RAZORPAY_KEY_SECRET
  "LangGraph": { "BaseUrl": "", "ApiKey": "" }, // env: LANGGRAPH_BASE_URL (Railway URL)
  "LiveKit": { "Url": "", "ApiKey": "", "ApiSecret": "" }, // env: LIVEKIT_URL / LIVEKIT_API_KEY /
LIVEKIT_API_SECRET
  "Cors": { "Origins": [ "https://setmycareer-counselor.vercel.app", "https://setmycareer.com" ] }
}

```

Complete key inventory across the system (names only):

Where	Keys
C# API (new)	SMC_DB_CONNECTION , SMC_JWT_SECRET , RAZORPAY_KEY_ID , RAZORPAY_KEY_SECRET , LANGGRAPH_BASE_URL , LIVEKIT_URL , LIVEKIT_API_KEY , LIVEKIT_API_SECRET
LangGraph service (Railway, existing)	GROQ_API_KEY , LLM_MODEL , OPENROUTER_API_KEY , OPENROUTER_MODEL , STT_MODEL , GOOGLE_CLIENT_ID/SECRET , ZOHO_CLIENT_ID/... , PORT
Vercel serverless (existing, keeps running)	GEMINI_API_KEY , GOOGLE_GENERATIVE_AI_API_KEY , GROQ_API_KEY , LIVEKIT_* , RAZORPAY_* , SUPABASE_URL , SUPABASE_KEY (retired at Phase 3), GOOGLE_ADS_*
Frontend (public by design)	VITE_SMC_WRITES_ENABLED , the Razorpay publishable key id only

Razorpay is on **TEST keys** today (doctrine: live keys are a deliberate, separate switch).

2.8 Security hardening (ship with, not after)

- **BCrypt** password hashing on the existing login tables (BACKEND_API_SPEC §3 — still the single most important item).
- JWT: existing logins (`LoginWithPassword` , `LoginWithOtp` , `NavigatorLogin` , `AdminLogin`) return a signed token `{uid, role}` ; all new endpoints require it. Reads that are open today stay open until the frontend sends tokens everywhere (two-phase, §2.10).
- CORS locked to the two app origins (`GET` , `POST` , `PUT` , `OPTIONS` + `Authorization`).
- Per-role row scoping as annotated in §2.5 — a client id in a token, never from a query param, is the ownership key.
- Rate-limit `Accounts/Create` , `SendOtp` , `GuestLinks/SubmitResult` (per-IP sliding window).
- Razorpay: server-side price catalog + HMAC verification; an unverified callback grants nothing.

2.9 IA ↔ API wiring map (the hand-in-hand table)

Touchpoint (Part 1)	Today	New endpoint(s)
Portal create account	local only	<code>Accounts/Create</code> , <code>Accounts/SendMagicLink</code>
Portal boot (who am I, revoked?)	localStorage	<code>Accounts/Me</code>
Profile intake save / gates	portal-store + cloud	<code>Profile/Upser</code> / <code>Profile/Get</code>
Test submit (all 4 engines)	results-store + cloud digest	<code>Tests/Submit</code>
Reports hub + counsellor/admin 360°	results-store	<code>Tests/ListMine</code> / <code>Tests/ListForClient</code>
Consolidated report	ad-hoc	<code>Synthesis/Latest</code> (auto-enqueued on battery completion)
Session request + auto-assign	bookings store + <code>assignment.ts</code>	<code>Sessions/Request</code>
Sessions page lists + recaps	bookings store	<code>Sessions/MyBookings</code> , <code>Sessions/Notes</code> , <code>Sessions/Transcript</code>
Transcript export / Compass hand-off	browser Blob + localStorage key	<code>Sessions/Transcript</code> (same payload feeds both)
Counsellor confirm/cancel/notes	stores	<code>`Sessions/Confirm</code>
Admin oversight panel	<code>useAllBookings</code>	<code>Admin/AllUpcomingBookings</code>
Admin reschedule	<code>rescheduleBooking</code>	<code>Sessions/Reschedule</code>
Admin revoke portal access	localStorage flag	<code>Admin/RevokePortalAccess</code> (cross-device at last)

Touchpoint (Part 1)	Today	New endpoint(s)
Counsellor agreement gate	localStorage	Counsellor/Agreement
Counsellor calendar blocks	calendar events	Counsellor/Blocks
Package & credits balances	wallet state	CreditLedger via Accounts/Me ; grants via Checkout/Verify
Checkout	/api/razorpay	Checkout/Create + Checkout/Verify (either fulfils)
Compass chats (all 3 doors)	Supabase app_chats	Chats/*
Client↔counsellor messages	cloud state	Messages/*
App state (bookings, calendar, wallet...)	Supabase app_state	State/*
Guest links mint / take / store	localStorage per browser	GuestLinks/*
Document uploads	live Reports/uploadReport	unchanged (already yours)

2.10 Migration order & go-live checklist

Phase 1 — stand up, dual-write (no user-visible change)

1. Create the DB tables (one EF migration), deploy the API, smoke `api/Health`.
2. Frontend: point `src/lib/cloud.ts` writes at BOTH `/api/cloud` (Supabase) and the new `State/Chats` endpoints (it is the single swap point — one file).
3. `Tests/Submit`, `Profile/Upsert`, `Sessions/*` dual-write the same way.

Phase 2 — flip reads + close the loops 4. Reads move to the C# API; verify parity on the three E2E logins (client 31369, counsellor 4104, admin) across: profile gate, battery submit → `Synthesis/Latest`, session request → auto-assign reason, admin reschedule → client re-confirm, revoke → cross-device sign-out, guest link → stored result. 5. Turn on JWT enforcement for writes; then reads.

Phase 3 — retire the interim 6. Supabase tables exported + archived (never deleted), `/api/cloud` returns 410, `SUPABASE_*` env vars removed. 7. Razorpay: swap TEST → LIVE keys (deliberate, separate change), ₹1 smoke purchase.

Definition of “live”: all §2.9 rows on the new API, the three E2E logins pass the Phase-2 parity list, `Synthesis/Latest` returns a stored report for a fresh battery, and the audit trail shows every admin action taken during the test pass.

2.11 What must NOT be built

- No `DELETE` verbs. No hard-delete admin tooling, “purge” scripts, or cascade deletes.
- No endpoint that accepts a price/amount from the client.
- No AI or payment secret in any response body, log line, or frontend bundle.
- No unversioned overwrite of `TestResult` / ledger rows — append or reject.

Part 3 – Testing recipe (proving “everything works immediately after it goes live”)

1. **Guest:** open `.../t/smcmain01` in a fresh browser → complete (or spot-check) both track branches → report renders → print PDF. `?only=` links isolate each instrument.
2. **Client:** create a fresh account (Part 0 demo box) → profile gate blocks tests/sessions → complete intake → battery unlocks → submit personality → report card appears → request a session → auto-assign toast names the counsellor + reason → Journey stream grows.
3. **Counsellor** (id 4104): agreement gate on first sign-in → caseload shows the client → confirm the booking → run the session → timestamped note + transcript → client’s recap gains Read/Export/Discuss-with-Compass.
4. **Admin:** oversight panel lists the booking with the assignment reason → reschedule it (client sees “awaiting confirm”) → audit trail logs it → revoke the demo client → their portal signs out → un-revoke.
5. **Money:** buy a credit pack with a Razorpay TEST card → `Checkout/Verify` grants credits → ledger shows `+N purchase` → Compass burns them.
6. **Synthesis:** after step 2’s third test, `Synthesis/Status` reaches `done` and the Reports hub renders the stored blueprint.

Steps 1–4 were executed against the current build on 18 Jul 2026 (see the audit log in the session notes); step 5 runs on TEST keys; step 6 goes live with §2.6.

Ship-readiness audit (18 Jul 2026) – every touchpoint swept, no empty feedback loops

A 43-agent audit (inventory of all five surfaces + adversarial verification of every claimed defect) ran against this build. **27 findings were confirmed and all were fixed** in the same release, including: the Billing “Choose one” dead link; Compass swallowing model errors (now an inline error row); the transcript→Compass hand-off now actually grounding the chat (context chip + prompt injection); silent OTP resend; misleading “magic link” sign-up copy; missing topbar titles; the counsellor topbar search now opening the ☿K palette; the AI ScheduleCard’s Confirm creating a **real** calendar event (or handing off to the scheduler); every Compass-bar quick action now navigating to its real destination or stating the truth; Settings’ “Export all data” producing a real JSON archive; the report builder’s export opening the real print/PDF path; the admin counsellor invite creating a **real** navigator account (temp password shown, activate via approval); the “Save to my plan” loop closed end-to-end (feedback

toast + a “Saved from Compass” pane on My journey); and the fake recording-bot switch replaced with the honest in-app-recording note. Orphaned screens were archived to `docs/attic/`. Verified by typecheck, production build, and live click-through before deploy.